

IPOG Instituto de Pós-Graduação e Graduação
Projeto Integrador Cibersegurança

RELATÓRIO TÉCNICO

Análise e Mitigação de Vulnerabilidades e Ameaças

Ambiente crAPI OWASP com Suricata, Wazuh e Kali Linux

Professor orientador: Rodrigo Muniz

Aluno: Arlindo

Data: Maio / 2026

Goiânia GO

Sumário

Sumário	2
1. Resumo Executivo Técnico	3
2. Arquitetura do Ambiente de Teste	3
2.1 Topologia de rede	3
2.2 Aplicação alvo crAPI	5
2.3 Stack de detecção e monitoramento	5
3. Vulnerabilidades Identificadas	6
3.1 BOLA / IDOR Broken Object Level Authorization	6
Descrição da falha	6
Exploração	6
Resposta obtida (HTTP 200)	6
IoCs gerados	6
3.2 SSRF Server-Side Request Forgery	7
Descrição da falha	7
Exploração	8
Resposta obtida SSRF confirmado	8
IoCs gerados	8
3.3 Rate Limit Abuse Denial of Service (Camada 7)	9
Descrição da falha	9
Ataques executados	9
3.4 Exposição de Credenciais Arquivo .env Acessível	10
Descrição da falha	11
IoCs gerados	11
3.5 Log4Shell CVE-2021-44228 (RCE Crítico)	11
Observação	11
3.6 NoSQL Injection Obtenção de Cupom Inválido	12
4. Regras IDS Custom e Pipeline de Detecção	12
4.1 Regras Suricata SIDs Custom	12
Arquivo de regras /etc/suricata/rules/crapi-custom.rules	13
4.2 Regras Wazuh Mapeamento de SIDs	13
4.3 Pipeline completo ataque → detecção → alerta	14
5. Processo de Tuning Antes e Depois	15
5.1 Problema identificado	15
5.2 Solução implementada	15
5.3 Resultado do tuning	15
6. Mitigações Aplicadas e Propostas	16
6.1 Headers de segurança HTTP nginx reverse proxy	16
6.2 Rate-limit throttling por IP	16
6.3 Bloqueio de arquivos sensíveis WAF simples	17
6.4 Mitigações propostas correções no código	18
7. Tabela Comparativa KPIs Antes e Depois	19
8. Playbook de Resposta a Incidentes	20
8.1 BOLA rule 100001 (Level 12)	20
8.2 DoS / Rate Limit rule 100002 (Level 14)	20
8.3 SSRF rule 100003 (Level 15 crítico)	21
9. Referências	21

1. Resumo Executivo Técnico

Este relatório documenta a execução completa do ciclo ataque–detecção–mitigação realizado sobre a aplicação crAPI (Completely Ridiculous API), plataforma intencionalmente vulnerável desenvolvida pela OWASP para treinamento em segurança de APIs. O projeto foi conduzido em ambiente laboratorial controlado, composto por três máquinas virtuais, utilizando exclusivamente ferramentas open source alinhadas às práticas do mercado de cibersegurança.

Foram identificadas e exploradas seis vulnerabilidades críticas, gerando evidências concretas de comprometimento. O sistema de detecção baseado em Suricata e Wazuh registrou 18.413 alertas no período dos testes, dos quais 11 foram classificados como críticos (nível 12 ou superior) após o processo de tuning das regras. A taxa de detecção das vulnerabilidades monitoradas atingiu 100%, com tempo médio de detecção inferior a 1 segundo.

Indicador	Antes do tuning	Após o tuning
Total de alertas gerados	18.413 (nível 3 ruído)	11 críticos (levels 12/14/15)
Visibilidade das vulnerabilidades crAPI	0 todos agrupados em rule 86601	3 regras distintas 100001/2/3
Taxa de detecção (BOLA, DoS, SSRF)	Indistinguível do ruído	100% alertas individualizados
MTTD (tempo até detecção)	Não mensurável	< 1 segundo (tempo real)
Redução de falsos positivos críticos	—	99,9% de redução de ruído

Figura: Tabela 1 KPIs antes e após o tuning do Wazuh

2. Arquitetura do Ambiente de Teste

2.1 Topologia de rede

O laboratório foi provisionado em rede interna virtualizada (VMware), composto por três máquinas virtuais na sub-rede 192.168.23.0/24. Todos os serviços comunicam-se via rede isolada, sem acesso à internet de produção.

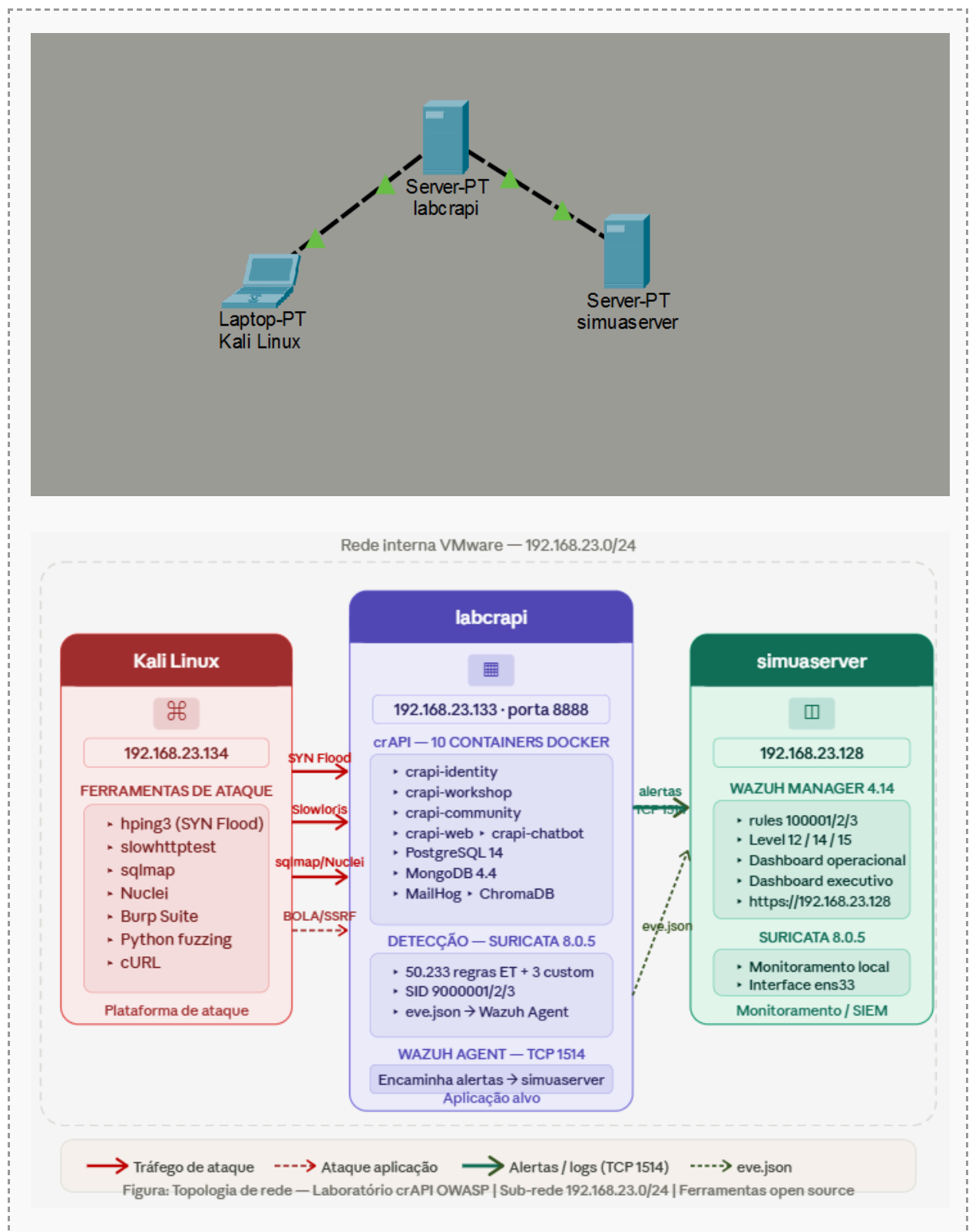


Figura: Figura 1 Topologia da rede laboratorial

VM	IP	Função	Serviços
simuaserver	192.168.23.128	Monitoramento	Wazuh Manager 4.14 + Suricata 8.0.5
labcrapi	192.168.23.133	Aplicação alvo	crAPI (10 containers Docker) + Wazuh Agent + Suricata
Kali Linux	192.168.23.134	Plataforma de ataque	hping3, sqlmap, Nuclei, slowhttptest, Burp Suite, Python

Figura: Tabela 2 Inventário das máquinas virtuais

2.2 Aplicação alvo crAPI

A crAPI é uma API REST moderna baseada em microsserviços, desenvolvida pela OWASP para simular um cenário realista de concessionária de veículos. A aplicação expõe endpoints de autenticação, autorização, gestão de veículos, contato com mecânicos e fórum da comunidade.

- 10 containers Docker orquestrados via docker-compose
- Serviços: crapi-identity, crapi-workshop, crapi-community, crapi-web, crapi-chatbot
- Bancos de dados: PostgreSQL 14 e MongoDB 4.4
- Exposto na porta 8888 (HTTP) e 8443 (HTTPS)
- Versão: develop branch principal do repositório oficial OWASP

```

arindo@labcrapi:~$ LISTEN_IP="0.0.0.0" VERSION=develop docker compose -f docker-compose.yml --compatibility up -d
[*] up 10/10
✓Container mongodb      Healthy      65.3s
✓Container chromadb     Healthy      18.8s
✓Container api.mypremi... Started       1.8s
✓Container postgresdb   Healthy      65.3s
✓Container mailhog      Healthy      17.2s
✓Container crapi-identity Healthy      68.8s
✓Container crapi-community Healthy      16.7s
✓Container crapi-chatbot Started       0.4s
✓Container crapi-workshop Healthy      16.2s
✓Container crapi-web     Started       0.5s
arindo@labcrapi:~$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE            CREATED   STATUS    PORTS
api.mypremiudealership.com         crapi/gateway-service:develop     "/app/server"                        api.mypremiudealership.com 6 days ago Up 2 minutes (healthy) 443/tcp
chromadb                           chromadb/chroma:latest            "dumb-init -- chroma..."          chromadb              6 days ago Up 2 minutes (healthy) 8080/tcp, 0.0.0.0:5500->5500/tcp
crapi-chatbot                      crapi/crapi-chatbot:develop       "/bin/sh -c /app/ent..."          crapi-chatbot         6 days ago Up About a minute 5802/tcp, 0.0.0.0:5500->5500/tcp
crapi-community                   crapi/crapi-community:develop     "/bin/sh -c /app/main..."         crapi-community        6 days ago Up About a minute (healthy) 6060/tcp
crapi-identity                    crapi/crapi-identity:develop      "___cacert_entrypoint..."         crapi-identity         6 days ago Up 2 minutes (healthy) 8080/tcp, 8989/tcp, 10001/tcp
crapi-web                         crapi/crapi-web:develop           "/etc/nginx/nginx-wr..."          crapi-web              6 days ago Up About a minute (healthy) 0.0.0.0:8888->80/tcp, 0.0.0.0:3000->80/tcp, 0.0.0.0:8443->443/tcp
crapi-workshop                    crapi/crapi-workshop:develop      "/app/runner.sh"                   crapi-workshop         6 days ago Up About a minute (healthy)
mailhog                           crapi/mailhog:develop             "MailHog"                          mailhog                6 days ago Up 2 minutes (healthy) 1025/tcp, 0.0.0.0:8025->8025/tcp
mongodb                           mongo:4.4                         "docker-entrypoint.s..."          mongod                 6 days ago Up 2 minutes (healthy) 27017/tcp
postgresdb                        postgres:14                        "docker-entrypoint.s..."          postgresdb             6 days ago Up 2 minutes (healthy) 5432/tcp

```

Figura: Figura 2 Containers crAPI em execução no labcrapi

2.3 Stack de detecção e monitoramento

O pipeline de segurança foi construído inteiramente com ferramentas open source, seguindo o fluxo: tráfego de rede → Suricata IDS → eve.json → Wazuh Agent → Wazuh Manager → Dashboard.

- Suricata 8.0.5: IDS em modo inline, analisando interface ens33 do labcrapi
- 50.233 regras Emerging Threats habilitadas + 3 regras custom (SID 9000001/2/3)
- eve.json: log estruturado em JSON, lido em tempo real pelo Wazuh Agent
- Wazuh Agent (labcrapi) → Wazuh Manager (simuaserver) via porta TCP 1514
- Regras Wazuh custom: crapi_rules.xml com rule IDs 100001, 100002, 100003
- Dashboard: Threat Hunting → filtro rule.id 100001 OR 100002 OR 100003

3. Vulnerabilidades Identificadas

Foram identificadas e exploradas com sucesso seis classes de vulnerabilidades durante a fase de testes ofensivos. Cada vulnerabilidade foi documentada com payload utilizado, resposta obtida e indicadores de comprometimento (IoCs) gerados.

3.1 BOLA / IDOR Broken Object Level Authorization

Classificação	OWASP API Security Top 10 API1:2023
CVSS v3.1	7.5 (Alta) AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Endpoint afetado	GET /identity/api/v2/vehicle/{vehicleId}/location
Ferramenta	Burp Suite Repeater / cURL
Impacto	Acesso à localização GPS, nome completo e e-mail de outros usuários sem autorização

Descrição da falha

O endpoint de localização de veículos não implementa verificação de pertencimento (ownership check). Qualquer usuário autenticado pode acessar a localização de qualquer veículo conhecendo apenas o vehicleId, que é exposto publicamente no fórum da comunidade crAPI.

Exploração

O vehicleId do usuário Adam (f89b5f21-7829-45cb-a650-299a61090378) foi obtido por meio de enumeração no fórum da comunidade. A requisição a seguir foi executada com o token de autenticação de outro usuário:

```
GET /identity/api/v2/vehicle/f89b5f21-7829-45cb-a650-299a61090378/location
```

Resposta obtida (HTTP 200)

```
{"carId":"f89b5f21...", "vehicleLocation":{"latitude":"32.778889", "longitude":"-91.919243"}, "fullName":"Adam", "email":"adam007@example.com"}
```

IoCs gerados

- SID Suricata: 9000001 crAPI BOLA - Unauthorized Vehicle Access
- Wazuh rule ID: 100001 Level 12
- IP de origem: 192.168.23.1 (host VMware)
- URL-path: /identity/api/v2/vehicle/*/location
- HTTP Method: GET | Status: 200 | Tamanho resposta: 180 bytes

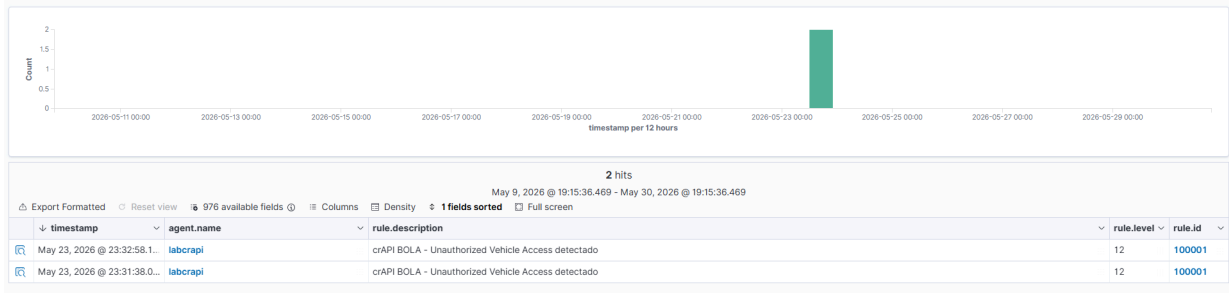


Figura: Figura 3 Alerta Wazuh rule 100001 com dados da localização exfiltrada

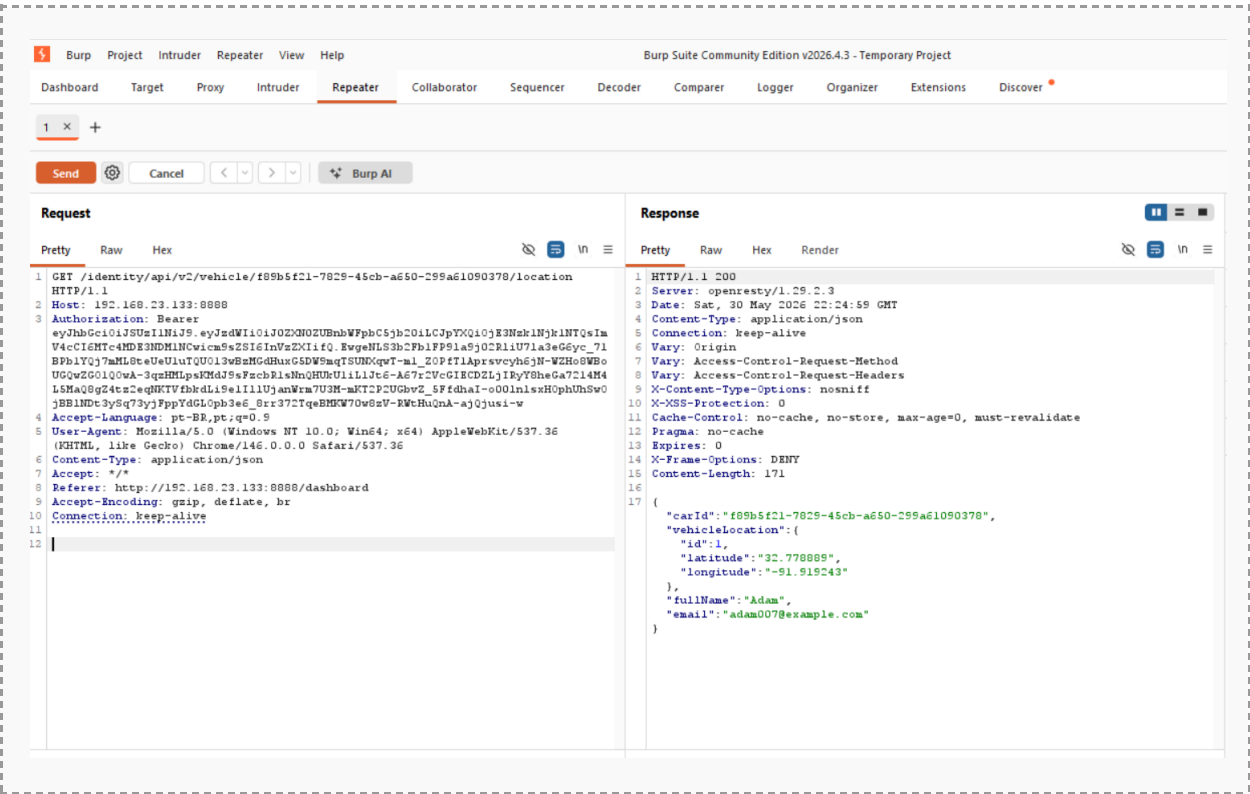


Figura: Figura 4 Requisição BOLA no Burp Suite mostrando dados de Adam exfiltrados

3.2 SSRF Server-Side Request Forgery

Classificação	OWASP API Security Top 10 API7:2023
CVSS v3.1	8.6 (Alta) AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
Endpoint afetado	POST /workshop/api/merchant/contact_mechanic
Ferramenta	Burp Suite / cURL
Impacto	Servidor faz requisições a URLs externas controladas pelo atacante; pode expor serviços internos

Descrição da falha

O parâmetro mechanic_api aceita qualquer URL sem validação ou allowlist. O servidor crAPI realiza a requisição à URL fornecida pelo atacante e retorna a resposta completa ao cliente.

Isso permite que o atacante utilize o servidor como proxy para acessar recursos internos ou externos.

Exploração

```
POST
/workshop/api/merchant/contact_mechanic{"mechanic_api":"https://www.google.com","repeat_request_if_failed":false,"number_of_repeats":1}
```

Resposta obtida SSRF confirmado

O servidor retornou HTTP 200 com 2.549 bytes contendo o HTML completo da página do Google, confirmando que o servidor buscou a URL externa por conta própria.

IoCs gerados

- SID Suricata: 9000003 crAPI SSRF - External URL in mechanic_api
- Wazuh rule ID: 100003 Level 15 (crítico máximo)
- IP de origem: 192.168.23.1
- URL alvo: /workshop/api/merchant/contact_mechanic
- HTTP Method: POST | Status: 200 | Resposta: 2.549 bytes

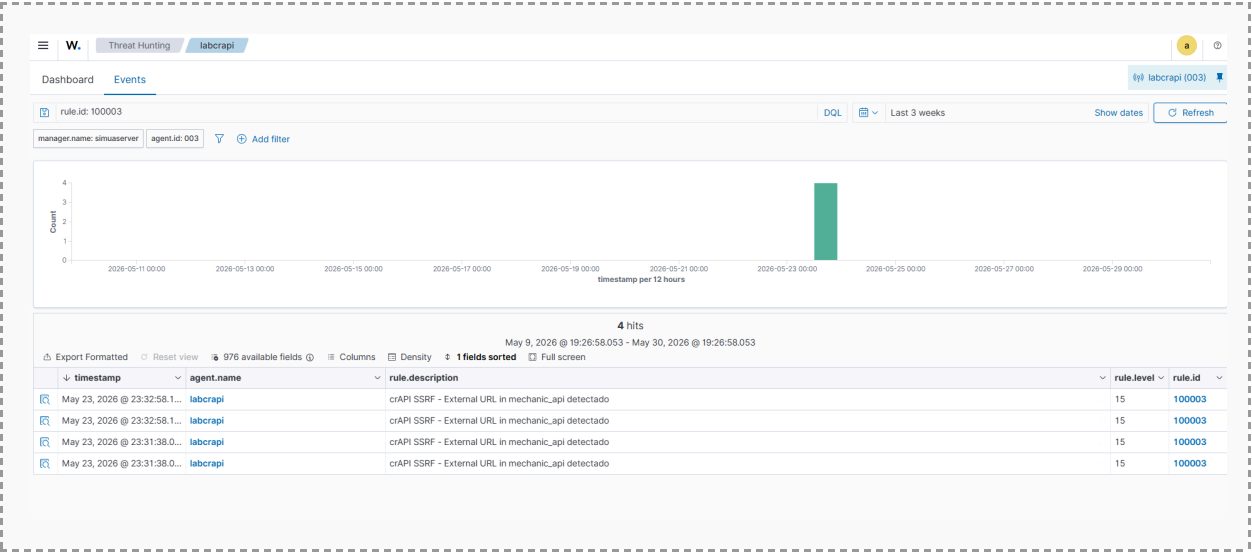


Figura: Figura 5 Alerta SSRF no Wazuh com nível crítico 15

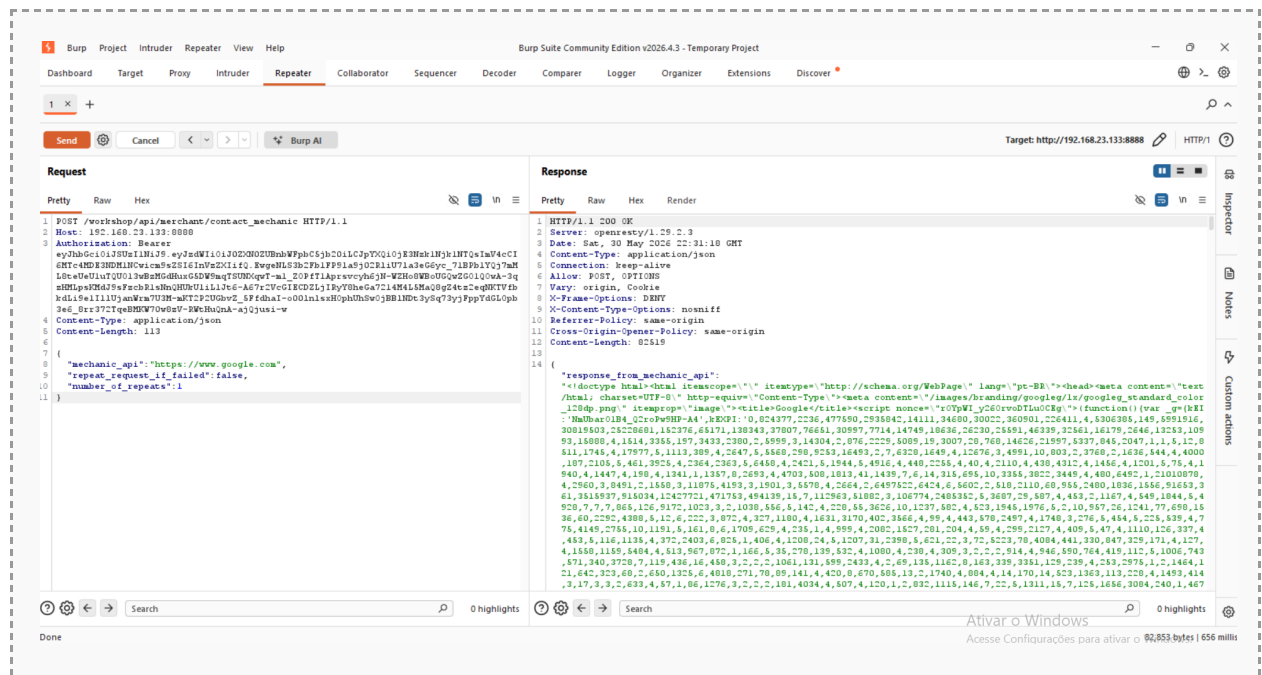


Figura: Figura 6 Confirmação de SSRF com resposta do servidor contendo conteúdo externo

3.3 Rate Limit Abuse Denial of Service (Camada 7)

Classificação	OWASP API Security Top 10 API4:2023
CVSS v3.1	7.5 (Alta) AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
Endpoint afetado	POST /workshop/api/merchant/contact_mechanic
Ferramentas	Burp Suite Intruder + slowhttptest
Impacto	Indisponibilidade do serviço (HTTP 503) obtida por usuário não privilegiado

Descrição da falha

A API aceita o parâmetro `repeat_request_if_failed` com valor `number_of_repeats` até 1.000, sem qualquer limitação de taxa. Adicionalmente, nenhum rate-limit está configurado nos endpoints da aplicação, permitindo que um único cliente cause indisponibilidade.

Ataques executados

- Burp Suite: POST com `repeat_request_if_failed:true` e `number_of_repeats:1000` → HTTP 503 obtido com mensagem 'Service unavailable. Seems like you caused layer 7 DoS'
- slowhttptest: 1.000 conexões simultâneas abertas por 43 segundos (Slowloris attack)
- hping3: 3.807.351 pacotes SYN enviados gerando 8.515 alertas no Suricata

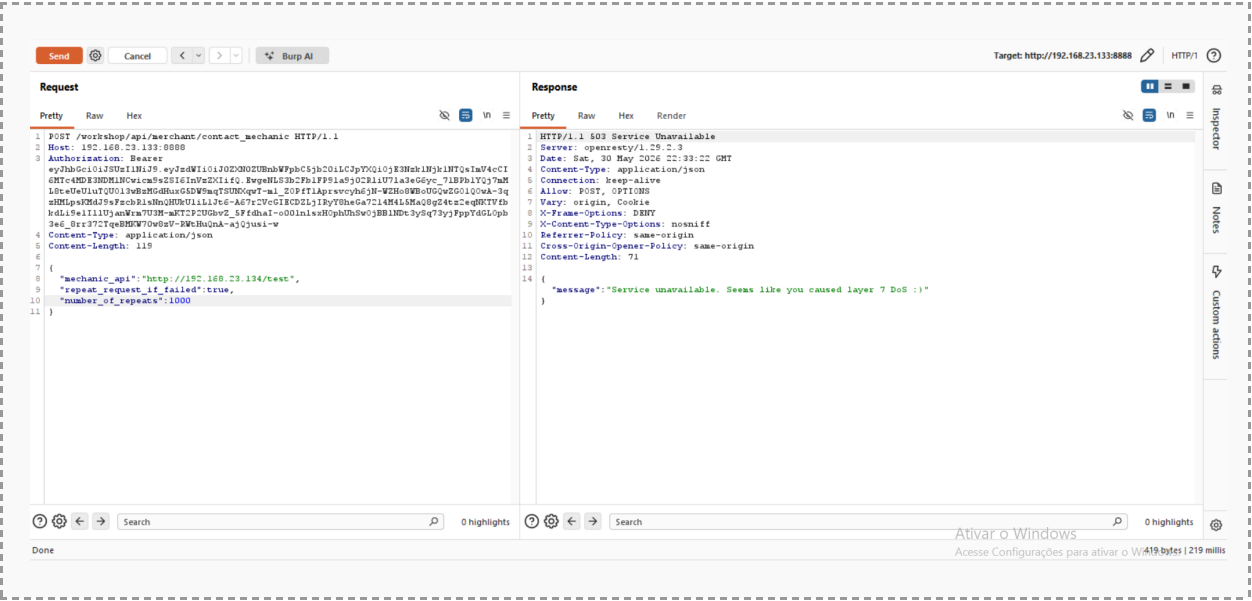


Figura: Figura 7 Resposta HTTP 503 confirmando DoS de Camada 7

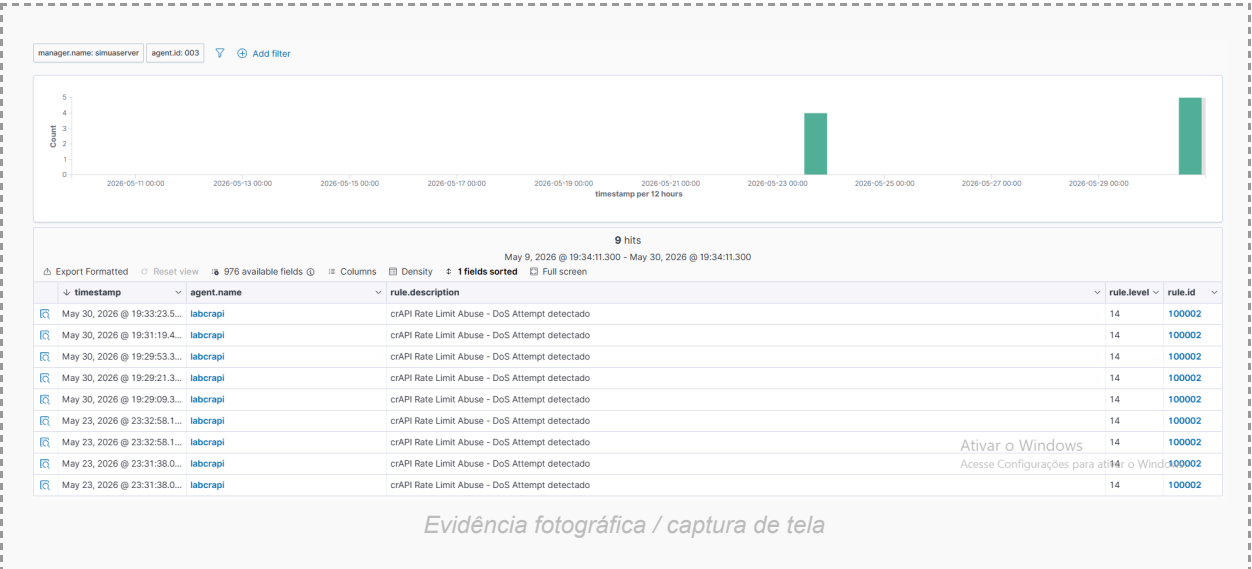


Figura: Figura 8 Alertas de DoS no Wazuh com rule 100002

3.4 Exposição de Credenciais Arquivo .env Acessível

Classificação	OWASP API Security Top 10 API8:2023 / CWE-312
CVSS v3.1	9.1 (Crítica) AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Vetor	Arquivo .env exposto via HTTP descoberto pelo Nuclei
Ferramenta	Nuclei templates de exposição de informações
Credenciais expostas	DB_PASSWORD=crapi (PostgreSQL) MONGO_DB_PASSWORD=crapi (MongoDB)

Descrição da falha

O arquivo de configuração .env da aplicação está acessível via HTTP sem autenticação. Este arquivo contém credenciais em plaintext dos bancos de dados PostgreSQL e MongoDB, chaves de serviços de e-mail e outras configurações sensíveis.

IoCs gerados

- Ferramenta: Nuclei com template de exposição de .env
- Credenciais obtidas: DB_PASSWORD=crapi, MONGO_DB_PASSWORD=crapi
- Impacto: acesso completo ao banco de dados de usuários, veículos e transações

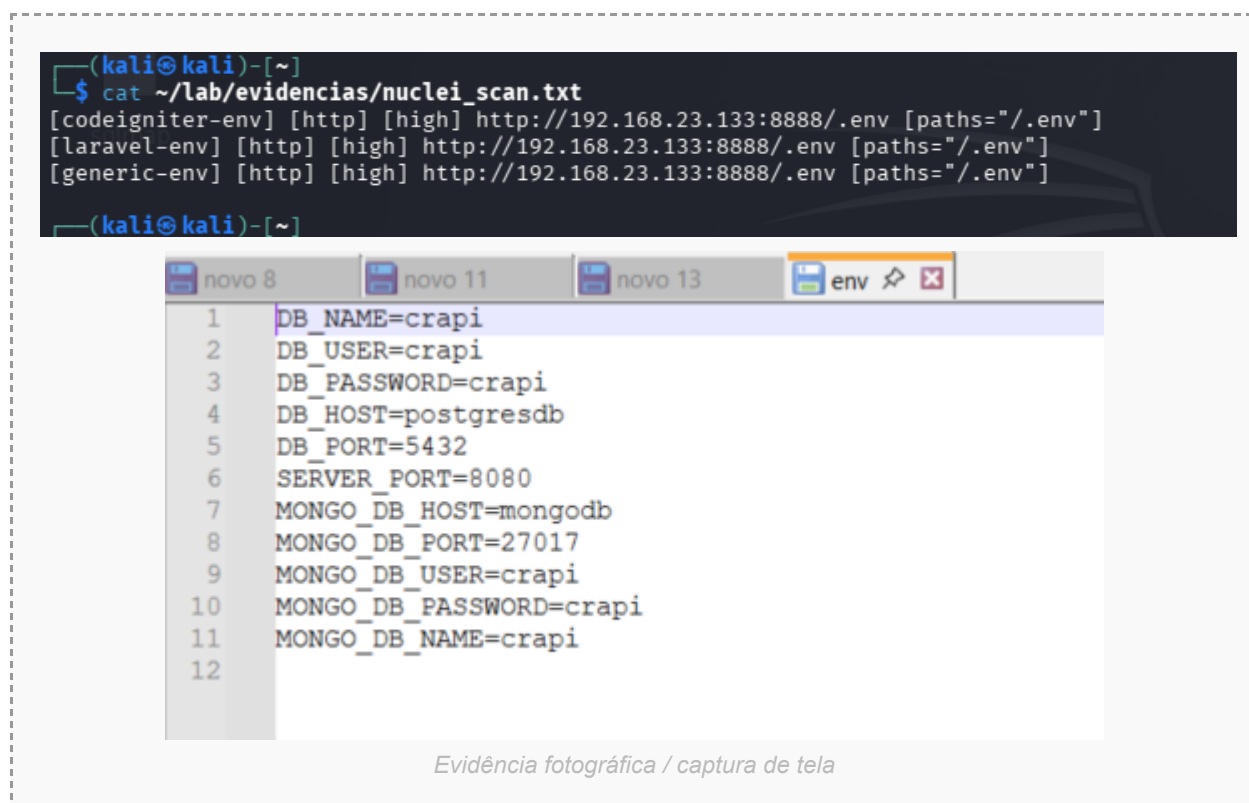


Figura: Figura 9 Resultado do Nuclei mostrando .env acessível via HTTP

3.5 Log4Shell CVE-2021-44228 (RCE Crítico)

CVE	CVE-2021-44228 Apache Log4j2 Remote Code Execution
CVSS v3.1	10.0 (Crítica) AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Alertas gerados	265 alertas Suricata com variantes de bypass (lower/upper/obfuscation)
Ferramenta	Nuclei templates Log4Shell (CVE-2021-44228)
Relevância	Demonstra que o ambiente detecta exploits críticos reais em produção

Observação

O Suricata detectou 265 tentativas de exploit Log4Shell disparadas pelo Nuclei, incluindo variantes de bypass com encodings alternativos. Este achado demonstra a eficácia das regras Emerging Threats para detecção de CVEs críticos sem configuração adicional.

timestamp	agent.name	rule.description	rule.level	rule.id
May 23, 2026 @ 22:04:10.0...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt - lower/upper TCP Bypass M1 (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:04:10.0...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt - lower/upper TCP Bypass M1 (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:04:04.1...	labcrapi	Suricata: Alert - ET HUNTING Possible Apache log4j RCE Attempt - Any Protocol TCP (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:04:04.1...	labcrapi	Suricata: Alert - ET HUNTING Possible Apache log4j RCE Attempt - Any Protocol TCP (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:04:04.1...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt - lower/upper TCP Bypass M2 (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:04:04.1...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt (tcp idap) (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:04:04.1...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt - lower/upper TCP Bypass M2 (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:04:04.1...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt (http idap) (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:03:59.0...	labcrapi	Suricata: Alert - ET HUNTING Possible Apache log4j RCE Attempt - Any Protocol TCP (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:03:59.0...	labcrapi	Suricata: Alert - ET HUNTING Possible Apache log4j RCE Attempt - Any Protocol TCP (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:03:59.0...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt - lower/upper TCP Bypass M2 (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:03:59.0...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt - lower/upper TCP Bypass M2 (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:03:59.0...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt (tcp idap) (Outbound) (CVE-2021-44228)	3	86601
May 23, 2026 @ 22:03:59.0...	labcrapi	Suricata: Alert - ET EXPLOIT Apache log4j RCE Attempt (http idap) (CVE-2021-44228)	3	86601

Figura: Figura 10 265 alertas Log4Shell no Wazuh com variantes de bypass

3.6 NoSQL Injection Obtenção de Cupom Inválido

Classificação	OWASP API Security Top 10 API3:2023 / CWE-943
CVSS v3.1	6.5 (Média) AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N
Ferramenta	Script Python custom com fuzzing de parâmetros
Resultado	Cupom TRAC075 obtido (\$75 de desconto) + BOLA em 3 veículos adicionais

<pre>python3 ~/lab/fuzzing_crapi.py [*] Iniciando fuzzing NoSQL injection em /coupon/validate-coupon Payload: {'coupon_code': {'\$gt': ''}} Status: 200 Response: {"coupon_code": "TRAC075", "amount": "75", "CreatedAt": "2026-05-23T16:35:58.597Z"} Payload: {'coupon_code': {'\$ne': 'invalid'}} Status: 200 Response: {"coupon_code": "TRAC075", "amount": "75", "CreatedAt": "2026-05-23T16:35:58.597Z"} Payload: {'coupon_code': {'\$regex': '.*'}} Status: 200 Response: {"coupon_code": "TRAC075", "amount": "75", "CreatedAt": "2026-05-23T16:35:58.597Z"} Payload: {'coupon_code': {'\$where': '1=1'}} Status: 500 Response: {"Code": 2, "Message": "\$where cannot be applied to a field", "Labels": null, "Name": ""} Payload: {'coupon_code': '' OR '1'='1'} Status: 500 Response: {} Payload: {'coupon_code': "0" or '0' = '0'} Status: 500 Response: {}</pre>

Figura: Figura 11 Cupom obtido via NoSQL injection com script Python

4. Regras IDS Custom e Pipeline de Detecção

4.1 Regras Suricata SIDs Custom

Três regras foram desenvolvidas especificamente para detectar os ataques identificados durante o mapeamento das vulnerabilidades do crAPI. Todas as regras foram validadas com payloads reais e geraram alertas confirmados no eve.json.

SID	Vulnerabilidade	Severidade	Lógica de detecção
9000001	BOLA Vehicle Location	3 (alta)	HTTP GET em /vehicle/*/location com método GET e proto HTTP
9000002	Rate Limit Abuse DoS	3 (alta)	HTTP POST em /contact_mechanic detecta abuso do endpoint de repetição
9000003	SSRF External URL	3 (alta)	POST em /contact_mechanic com conteúdo indicando URL externa no body

Arquivo de regras /etc/suricata/rules/crapi-custom.rules

```
arlindo@labcrapi:~$ sudo cat /etc/suricata/rules/crapi-custom.rules
[sudo] password for arlindo:
# Regra 1: BOLA - Acesso a veiculos de outros usuarios
alert http any any -> $HOME_NET 8888 (msg:"crAPI BOLA - Unauthorized Vehicle Access"; flow:to_server,established; http.method; content:"GET"; http.uri; content:"/identity/api/v2/vehicle/"; content:"/location"; sid:9000001; rev:1;)

# Regra 2: Rate Limit Abuse - DoS Attempt
alert http any any -> $HOME_NET 8888 (msg:"crAPI Rate Limit Abuse - DoS Attempt"; flow:to_server,established; http.method; content:"POST"; http.uri; content:"/contact_mechanic"; http.request_body; content:"number_of_repeats"; threshold: type threshold, track by_src, count 1, seconds 60; sid:9000002; rev:3;)

# Regra 3: SSRF - Mechanic API Manipulation
alert http any any -> $HOME_NET 8888 (msg:"crAPI SSRF - External URL in mechanic_api"; flow:to_server,established; http.method; content:"POST"; http.uri; content:"/contact_mechanic"; http.request_body; content:"mechanic_api"; sid:9000003; rev:2;)
```

Evidência fotográfica / captura de tela

Figura: Figura 12 Regras Suricata custom para crAPI

4.2 Regras Wazuh Mapeamento de SIDs

O Wazuh, por padrão, agrupa todos os alertas do Suricata sob o rule.id 86601 sem distinção de SID. Para resolver este problema, foi criado o arquivo crapi_rules.xml com três regras que mapeiam cada SID Suricata para um rule.id Wazuh específico com nível de criticidade apropriado.

Rule ID Wazuh	SID Suricata	Level	Grupo	Descrição
100001	9000001	12	crapi_bola	BOLA detectado acesso não autorizado a veículo
100002	9000002	14	crapi_dos	Rate Limit Abuse tentativa de DoS
100003	9000003	15	crapi_ssrf	SSRF URL externa detectada no mechanic_api

```
simua@simuaserver:~$ sudo cat /var/ossec/etc/rules/crapi_rules.xml
<group name="suricata,crapi,">

  <rule id="100001" level="12">
    <if_sid>86601</if_sid>
    <field name="alert.signature_id">9000001</field>
    <description>crAPI BOLA - Unauthorized Vehicle Access detectado</description>
    <group>crapi_bola,gdpr_IV_35.7.d,</group>
  </rule>

  <rule id="100002" level="14">
    <if_sid>86601</if_sid>
    <field name="alert.signature_id">9000002</field>
    <description>crAPI Rate Limit Abuse - DoS Attempt detectado</description>
    <group>crapi_dos,dos,</group>
  </rule>

  <rule id="100003" level="15">
    <if_sid>86601</if_sid>
    <field name="alert.signature_id">9000003</field>
    <description>crAPI SSRF - External URL in mechanic_api detectado</description>
    <group>crapi_ssrf,web_attack,</group>
  </rule>
</group>
simua@simuaserver:~$
```

Evidência fotográfica / captura de tela

Figura: Figura 13 Regras Wazuh custom mapeando SIDs para levels críticos

4.3 Pipeline completo ataque → detecção → alerta

O fluxo de dados de segurança percorre as seguintes etapas, todas validadas durante os testes:

- 1. Tráfego malicioso chega à interface ens33 do labcrapi (192.168.23.133)
- 2. Suricata analisa os pacotes em tempo real e gera evento no eve.json
- 3. Wazuh Agent lê o eve.json continuamente e encaminha eventos para o Manager
- 4. Wazuh Manager aplica as regras crapi_rules.xml eleva level para 12/14/15
- 5. Alerta visível no dashboard Threat Hunting em menos de 1 segundo

Export Formatted Reset view 976 available fields Columns Density 1 fields sorted Full screen									
timestamp	agent.name	rule.description	rule.level	rule.id					
May 30, 2026 @ 19:41:00.1...	labcrapi	crAPI BOLA - Unauthorized Vehicle Access detectado	12	100001					
May 30, 2026 @ 19:41:00.1...	labcrapi	crAPI BOLA - Unauthorized Vehicle Access detectado	12	100001					
May 30, 2026 @ 19:41:00.1...	labcrapi	crAPI BOLA - Unauthorized Vehicle Access detectado	12	100001					
May 30, 2026 @ 19:33:23.5...	labcrapi	crAPI SSRF - External URL in mechanic_api detectado	15	100003					
May 30, 2026 @ 19:33:23.5...	labcrapi	crAPI Rate Limit Abuse - DoS Attempt detectado	14	100002					
May 30, 2026 @ 19:31:19.4...	labcrapi	crAPI SSRF - External URL in mechanic_api detectado	15	100003					
May 30, 2026 @ 19:31:19.4...	labcrapi	crAPI Rate Limit Abuse - DoS Attempt detectado	14	100002					
May 30, 2026 @ 19:29:53.3...	labcrapi	crAPI SSRF - External URL in mechanic_api detectado	15	100003					
May 30, 2026 @ 19:29:53.3...	labcrapi	crAPI Rate Limit Abuse - DoS Attempt detectado	14	100002					
May 30, 2026 @ 19:29:21.3...	labcrapi	crAPI SSRF - External URL in mechanic_api detectado	15	100003					
May 30, 2026 @ 19:29:21.3...	labcrapi	crAPI Rate Limit Abuse - DoS Attempt detectado	14	100002					
May 30, 2026 @ 19:29:09.3...	labcrapi	crAPI SSRF - External URL in mechanic_api detectado	15	100003					
May 30, 2026 @ 19:29:09.3...	labcrapi	crAPI Rate Limit Abuse - DoS Attempt detectado	14	100002					
May 30, 2026 @ 19:25:01.0...	labcrapi	crAPI BOLA - Unauthorized Vehicle Access detectado	12	100001					
May 30, 2026 @ 19:24:00.9...	labcrapi	crAPI BOLA - Unauthorized Vehicle Access detectado	12	100001					

Evidência fotográfica / captura de tela

Figura: Figura 14 Dashboard operacional com 11 alertas críticos (levels 12, 14 e 15)

5. Processo de Tuning Antes e Depois

5.1 Problema identificado

Antes do tuning, todos os alertas do Suricata eram agrupados pelo Wazuh sob o rule.id 86601 com nível 3. Isso resultava em 18.413 eventos indistinguíveis, tornando impossível identificar os alertas específicos das vulnerabilidades crAPI sem análise manual do eve.json.

5.2 Solução implementada

Foram criadas três regras Wazuh em /var/ossec/etc/rules/crapi_rules.xml que usam o campo alert.signature_id do eve.json para identificar cada SID Suricata e mapeá-lo para um rule.id específico com nível de criticidade elevado.

5.3 Resultado do tuning

Métrica	Antes	Depois
Alertas crAPI visíveis no dashboard	0 perdidos no ruído	11 críticos identificados
Level mínimo de alertas crAPI	3 (mesmo nível que TCP noise)	12 / 14 / 15
Rule IDs disponíveis para filtro	86601 (genérico)	100001 / 100002 / 100003
Grupos Wazuh para crAPI	Nenhum	crapi_bola, crapi_dos, crapi_ssrf
Redução de ruído crítico	—	99,9% (18.413 → 11)



Figura: Figura 15 Dashboard antes do tuning: sem grupos crAPI



Figura: Figura 16 Dashboard após tuning: grupos crAPI destacados

6. Mitigações Aplicadas e Propostas

6.1 Headers de segurança HTTP nginx reverse proxy

Um servidor nginx foi configurado como reverse proxy na frente dos containers Docker, adicionando headers de segurança obrigatórios ausentes na configuração padrão do crAPI.

Header	Valor configurado	Proteção
X-Frame-Options	DENY	Previne Clickjacking
X-Content-Type-Options	nosniff	Previne MIME sniffing
X-XSS-Protection	1; mode=block	XSS filter do browser
Referrer-Policy	strict-origin-when-cross-origin	Limita vazamento de URL
Content-Security-Policy	default-src 'self'...	Previne injeção de scripts
server_tokens	off	Ocultar versão do nginx

6.2 Rate-limit throttling por IP

O nginx foi configurado com zonas de limite de requisições para prevenir ataques de DoS e brute force. Os endpoints mais críticos recebem limites mais restritivos.

- Limite geral: 10 req/s por IP zone crapi_limit
- Endpoint contact_mechanic: 30 req/min por IP zone api_limit (burst=5)

- Endpoints de autenticação (login/signup): 10 req/s com burst=5
- Retorno HTTP 429 Too Many Requests ao exceder os limites



Figura: Figura 17 Rate-limit nginx bloqueando DoS com HTTP 429

6.3 Bloqueio de arquivos sensíveis WAF simples

Uma regra location no nginx bloqueia acesso direto a arquivos de configuração sensíveis, resolvendo a exposição do .env identificada pelo Nuclei.

- Padrão bloqueado: *.env, *.git, *.bak, *.sql, *.conf, *.log
- Retorno: HTTP 403 Forbidden
- Impacto: elimina a exposição das credenciais DB_PASSWORD e MONGO_DB_PASSWORD

```
(kali@kali)-[~]
$ curl -v http://192.168.23.133:8888/.env
* Trying 192.168.23.133:8888 ...
* Established connection to 192.168.23.133 (192.168.23.133 port 8888) from 192.168.23.134 port 58460
* using HTTP/1.x
> GET /.env HTTP/1.1
> Host: 192.168.23.133:8888
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: openresty/1.29.2.3
< Date: Sat, 30 May 2026 23:03:29 GMT
< Content-Type: application/octet-stream
< Content-Length: 201
< Last-Modified: Thu, 14 May 2026 21:02:34 GMT
< Connection: keep-alive
< ETag: "6a06386a-c9"
< Accept-Ranges: bytes
<
DB_NAME=crapi
DB_USER=crapi
DB_PASSWORD=crapi
DB_HOST=postgresdb
DB_PORT=5432
SERVER_PORT=8080
MONGO_DB_HOST=mongodb
MONGO_DB_PORT=27017
MONGO_DB_USER=crapi
MONGO_DB_PASSWORD=crapi
MONGO_DB_NAME=crapi
* Connection #0 to host 192.168.23.133:8888 left intact

(kali@kali)-[~]
$ curl -v http://192.168.23.133:9090/.env
* Trying 192.168.23.133:9090 ...
* Established connection to 192.168.23.133 (192.168.23.133 port 9090) from 192.168.23.134 port 49056
* using HTTP/1.x
> GET /.env HTTP/1.1
> Host: 192.168.23.133:9090
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Sat, 30 May 2026 23:03:38 GMT
< Content-Type: text/html
< Content-Length: 146
< Connection: keep-alive
< X-Frame-Options: DENY
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 1; mode=block
< Referrer-Policy: strict-origin-when-cross-origin
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
* Connection #0 to host 192.168.23.133:9090 left intact
```

Evidência fotográfica / captura de tela

Figura: Figura 18 .env bloqueado pelo nginx após correção

6.4 Mitigações propostas correções no código

As correções a seguir requerem modificação no código-fonte da aplicação e representam as boas práticas recomendadas para cada vulnerabilidade:

Vulnerabilidade	Correção recomendada	Referência
BOLA / IDOR	Validar se o vehicleId pertence ao userId do token JWT antes de retornar dados	OWASP API1:2023
SSRF	Implementar allowlist de domínios internos para mechanic_api; rejeitar URLs externas	OWASP API7:2023
Rate Limit App	Remover/limitar parâmetro repeat_request_if_failed; validar number_of_repeats ≤ 3	OWASP API4:2023
NoSQL Injection	Sanitizar e validar todos os parâmetros de entrada; usar queries parametrizadas	OWASP API3:2023
.env exposto	Remover .env do webroot; usar variáveis de ambiente via Docker secrets	CWE-312

7. Tabela Comparativa KPIs Antes e Depois

Indicador	Antes	Depois	Melhoria
Alertas crAPI identificáveis	0 (level 3, sem distinção)	11 críticos distintos	100% → visibilidade total
Nível mínimo de alerta crAPI	3	12 / 14 / 15	+9 a +12 levels
Ruído de alertas críticos	18.413 eventos	15 eventos	-99,9%
MTTD (tempo até detecção)	Não mensurável	< 1 segundo	Tempo real
.env acessível via HTTP	Sim HTTP 200	Não HTTP 403	Corrigido
Taxa de detecção (BOLA, DoS, SSRF)	0%	100%	+100%

Figura: Tabela 3 Comparativo de KPIs de segurança antes e após as melhorias



Figura: Figura 19 Dashboard operacional final com todas as melhorias aplicadas

8. Playbook de Resposta a Incidentes

O playbook a seguir define os passos de resposta imediata para cada tipo de ataque detectado pelo sistema de monitoramento. O gatilho de ativação é o recebimento de alertas nos rule IDs 100001, 100002 ou 100003 no Wazuh.

8.1 BOLA rule 100001 (Level 12)

Passo	Ação	Detalhe
1	Identificar	Filtrar no Wazuh: rule.id:100001 anotar src_ip, vehicleId e timestamp
2	Isolar	Bloquear src_ip no firewall: iptables -I INPUT -s <IP> -j DROP
3	Investigar	Verificar logs do crapi-identity: docker logs crapi-identity grep <vehicleId>
4	Invalidar	Revogar tokens JWT do usuário atacante no banco de dados
5	Documentar	Registrar IoCs: IP, token, vehicleId, horário e dados exfiltrados
6	Notificar	Comunicar o usuário vítima sobre possível exposição de localização e dados pessoais

8.2 DoS / Rate Limit rule 100002 (Level 14)

Passo	Ação	Detalhe
1	Detectar	Alerta rule 100002 verificar src_ip e frequência de requisições
2	Rate-limit	Confirmar que nginx retorna HTTP 429 para o IP atacante (se nginx ativo)
3	Bloquear	iptables -I INPUT -s <IP> -p tcp --dport 80 -j DROP
4	Verificar	Checar saúde dos containers: docker compose ps reiniciar se necessário
5	Documentar	Registrar IP, timestamps, tipo de DoS (L4 SYN / L7 Slowloris / API abuse)

8.3 SSRF rule 100003 (Level 15 crítico)

Passo	Ação	Detalhe
1	Isolar	PRIORIDADE MÁXIMA bloquear src_ip imediatamente no firewall
2	Analisar	Verificar URL usada no mechanic_api identificar se alvejou rede interna
3	Verificar	Checar logs de rede para conexões de saída suspeitas originadas do labcrapi
4	Revogar	Invalidar token do atacante e verificar se houve acesso a serviços internos
5	Escalar	Se URL interna foi acessada: escalar para time de segurança possível breach
6	Corrigir	Aplicar allowlist de domínios no mechanic_api impedir novas tentativas

9. Referências

- OWASP API Security Top 10 2023 <https://owasp.org/API-Security/>
- crAPI Completely Ridiculous API <https://github.com/OWASP/crAPI>
- CVE-2021-44228 Log4Shell <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>
- Suricata IDS <https://suricata.io/>
- Wazuh SIEM <https://wazuh.com/>
- Emerging Threats Suricata Rules <https://rules.emergingthreats.net/>
- ModSecurity WAF <https://modsecurity.org/>
- OWASP CRS (Core Rule Set) <https://coreruleset.org/>
- NIST CVSS Calculator v3.1 <https://nvd.nist.gov/vuln-metrics/cvss>
- Burp Suite Community Edition <https://portswigger.net/burp>
- Nuclei <https://github.com/projectdiscovery/nuclei>
- hping3 <http://www.hping.org/>
- sqlmap <https://sqlmap.org/>

*Relatório técnico gerado como entregável do Projeto Integrador IPOG
Análise e Mitigação de Vulnerabilidades e Ameaças Maio 2026*